


**University of Moratuwa, Sri Lanka**

Department of Electronic & Telecommunication Engineering

# EN3150 Pattern Recognition

## Assignment 03

Resource-Constrained CNN for
Edge Image Classification

**Group PatternX**

| Index number | Name |
|---|---|
| 220303E | Kariyawasam U.G.R.T. |
| 220370E | Maddumage M.M.T.D. |
| 220522A | Rathnayaka R.M.P.B. |
| 220525K | Rathnayake R.M.D.D. |

Source code and experiment pipeline:
`github.com/Parawork/CNN-for-image-classification`

October 19, 2026

# Contents


**1 Data Preparation** **1**

1.1 Objective and dataset choice . . . . . 1

1.2 Splits and preprocessing . . . . . 1

**2 Custom Architecture Design** **3**

2.1 Shared topology and parameter accounting . . . . . 3

2.2 Model A: standard convolutions . . . . . 3

2.3 Model B: depthwise separable convolutions . . . . . 4

2.4 Hardware-aware design choices . . . . . 5

**3 Optimizer Selection and Tuning** **6**

3.1 Experimental protocol . . . . . 6

3.2 Selected optimizer and learning rate . . . . . 6

3.3 Learning-rate sensitivity and momentum . . . . . 7

3.4 Scope of the optimizer comparison . . . . . 9

**4 Custom Model Training and Evaluation** **10**

4.1 Training settings and checkpoint selection . . . . . 10

4.2 Loss curves and convergence . . . . . 10

4.3 Metrics and the custom-model comparison . . . . . 11

4.4 Effect of depthwise separation . . . . . 14

**5 Lightweight SOTA Fine-Tuning and Evaluation** **15**

5.1 Architectures and fine-tuning procedure . . . . . 15

5.2 Test performance and learning curves . . . . . 15

**6 Final Comparison and Trade-off Discussion** **19**

6.1 Accuracy, storage and computational cost . . . . . 19

6.2 Why smaller operation counts do not settle the choice . . . . . 20

6.3 Custom training versus pretrained networks . . . . . 20

6.4 Deployment and experimental limitations . . . . . 21

**A Reproducibility and Result Provenance** **22**

**References** **23**

# List of Figures


1 One RGB example from each EuroSAT class at the input resolution used by every network. . . . . 1

2 Training and validation loss for Adam, plain SGD and SGD with momentum 0.9. Original recorded plot panels are stacked for legibility. . . . . 7

3 Adam learning-rate sweep on Model B. All trials use the same initial weights and 20-epoch budget. . . . . 8

4 Effect of momentum with SGD learning rate fixed at $10^{-2}$. . . . . 9

5 Model A training and validation cross-entropy over 30 epochs. Test metrics use the best validation checkpoint, not necessarily the final epoch. . . . . 11

6 Model B training and validation cross-entropy over 30 epochs. . . . . 11

7 Model A test confusion matrix, 4,050 images. . . . . 13

8 Model B test confusion matrix, using exactly the same test indices as Model A. 14

9 MobileNetV2 fine-tuning loss over 20 epochs. The selected checkpoint is from epoch 15. . . . . 16

10 ShuffleNetV2 fine-tuning loss over 20 epochs. The selected checkpoint is from epoch 19. . . . . 16

11 MobileNetV2 test confusion matrix after fine-tuning. . . . . 17

12 ShuffleNetV2 test confusion matrix after fine-tuning. . . . . 18

13 Measured accuracy versus float32 checkpoint size. Bubble area is proportional to convolutional and linear MACs; labels show millions of MACs. . . . . 19

# List of Tables


1 Class distribution in the shared 70/15/15 split. . . . . 2

2 Model A architecture; shapes are after the operation indicated. . . . . 4

3 Model B block parameters. Both BatchNorm layers are included in each block's parameter count. . . . . 4

4 Model B optimizer study, 20 epochs per run. $L_v$ is validation cross-entropy; $\sigma_{v,5}$ is its standard deviation over epochs 16–20. A dash means 85% was not reached. . . . . 6

5 Training budgets and checkpoint selection for the four final models. . . . . 10

6 Required Model A versus Model B comparison, using measured float32 checkpoint sizes. . . . . 12

7 Custom-model per-class precision (P) and recall (R), in percent. . . . . 12

8 Best-checkpoint test performance. Precision and recall are macro averages. . . 15

9 Pretrained-model per-class precision (P) and recall (R), in percent. . . . . 16

10 Resources at $64 \times 64$ input. CPU latency is single-thread, batch one; training time is the mean training-only epoch duration on the GPU. . . . . 19

# 1. Data Preparation

## *1.1 Objective and dataset choice*

This assignment investigates the accuracy, storage and computational costs of image classification under a limited resource budget. Two custom networks are compared: a conventional convolutional network (Model A) and a depthwise separable network (Model B) with fewer than 100,000 trainable parameters. The same task is then used to fine-tune two pretrained mobile architectures, MobileNetV2 and ShuffleNetV2.

We use the RGB version of EuroSAT, a Sentinel-2 land-use and land-cover dataset with 27,000 labelled images across ten classes [1], illustrated in Figure 1. Classifying these images on an edge node could reduce image transmission. The RGB images are natively $64 \times 64$ pixels. The loader explicitly resizes them to $64 \times 64$, satisfying the assignment's resolution limit; no CIFAR-10 data are used. All four networks receive the same $3 \times 64 \times 64$ inputs. The optional larger-input experiment supported by the code is not used in these results.

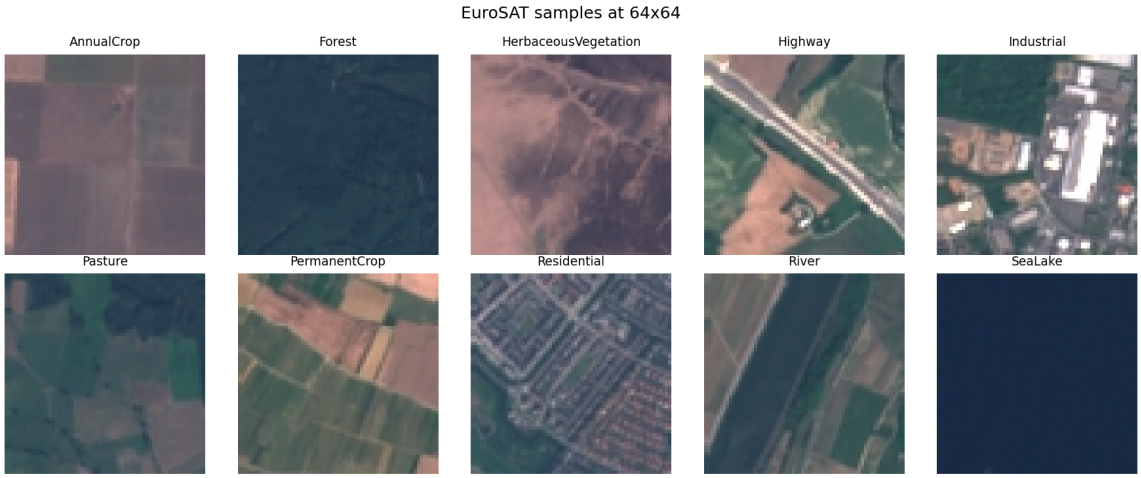



**Figure 1:** One RGB example from each EuroSAT class at the input resolution used by every network.

## *1.2 Splits and preprocessing*

We first reserve 30% of the indices with a stratified split, then split that portion equally into validation and test sets, again using stratification. Both splits use seed 42. This gives 18,900 training images, 4,050 validation images and 4,050 test images (Table 1). The saved `data/splits.json` contains the indices reused by every experiment. A check confirms that the three sets do not overlap and together contain all 27,000 indices exactly once.

**Table 1:** Class distribution in the shared 70/15/15 split.

| Class | Train | Validation | Test | Total |
|---|---|---|---|---|
| AnnualCrop | 2,100 | 450 | 450 | 3,000 |
| Forest | 2,100 | 450 | 450 | 3,000 |
| HerbaceousVegetation | 2,100 | 450 | 450 | 3,000 |
| Highway | 1,750 | 375 | 375 | 2,500 |
| Industrial | 1,750 | 375 | 375 | 2,500 |
| Pasture | 1,400 | 300 | 300 | 2,000 |
| PermanentCrop | 1,750 | 375 | 375 | 2,500 |
| Residential | 2,100 | 450 | 450 | 3,000 |
| River | 1,750 | 375 | 375 | 2,500 |
| SeaLake | 2,100 | 450 | 450 | 3,000 |
| **Total** | 18,900 | 4,050 | 4,050 | 27,000 |

RGB values are converted from unsigned bytes to floating-point values in $[0, 1]$. Each channel is then normalized as

$$x'_c = \frac{x_c - \mu_c}{\sigma_c}, \qquad \boldsymbol{\mu} = (0.485, 0.456, 0.406), \quad \boldsymbol{\sigma} = (0.229, 0.224, 0.225).$$

These are the ImageNet normalization constants used for the pretrained models; the same transformation is applied to the custom networks. The training set alone receives random horizontal and vertical flips, each with probability 0.5, and a rotation selected uniformly from $0°$, $90°$, $180°$ and $270°$. Such orientation changes are appropriate for overhead land-cover patches. Validation and test images receive no random augmentation.

Class sizes range from 2,000 to 3,000 images. Stratification preserves these proportions. We report macro precision and recall so that the smaller Pasture class receives equal weight in these measures. This random image split measures generalization to held-out patches from this dataset; it does not demonstrate generalization to a geographically separate region.

# 2. Custom Architecture Design

## *2.1 Shared topology and parameter accounting*

Both models use channel widths $32, 64, 128, 256$, four $2 \times 2$ max-pooling layers, global average pooling (GAP), a 64-unit fully connected layer and a ten-unit output layer. Pooling reduces spatial size as $64 \to 32 \to 16 \to 8 \to 4$. The hidden classifier uses ReLU and dropout with probability 0.3. The output contains unnormalized logits; cross-entropy is used during training and the largest logit determines the predicted class.

All convolutional layers use no bias because BatchNorm follows them. BatchNorm contributes two trainable values per channel, its scale and shift; running means, variances and counters are buffers rather than trainable parameters. For kernel width $k$, input channels $C_{\text{in}}$ and output channels $C_{\text{out}}$,

$$P_{\text{standard}} = k^2 C_{\text{in}} C_{\text{out}}, \tag{1}$$

$$P_{\text{depthwise+pointwise}} = k^2 C_{\text{in}} + C_{\text{in}} C_{\text{out}}, \tag{2}$$

$$P_{\text{FC}} = n_{\text{in}} n_{\text{out}} + n_{\text{out}}. \tag{3}$$

For output spatial size $H \times W$, convolutional MAC counts equal the convolutional weight count multiplied by $HW$. A fully connected layer contributes $n_{\text{in}} n_{\text{out}}$ MACs. Counts below include convolutions and linear layers; they exclude normalization, activation, pooling and memory movement. One multiply-accumulate is counted as one MAC.

## *2.2 Model A: standard convolutions*

Each feature block is a $3 \times 3$ convolution with stride 1 and padding 1, followed by BatchNorm, ReLU and max-pooling. Table 2 gives the complete trainable parameter calculation. Pooling, GAP, ReLU and dropout have no trainable parameters.

**Table 2:** Model A architecture; shapes are after the operation indicated.

| Operation | Output shape | Parameters | MACs |
|---|---|---|---|
| Conv $3\times3$, $3 \to 32$ + BN + ReLU | $64 \times 64 \times 32$ | 928 | 3,538,944 |
| MaxPool $2 \times 2$ | $32 \times 32 \times 32$ | 0 | – |
| Conv $3\times3$, $32 \to 64$ + BN + ReLU | $32 \times 32 \times 64$ | 18,560 | 18,874,368 |
| MaxPool $2 \times 2$ | $16 \times 16 \times 64$ | 0 | – |
| Conv $3\times3$, $64 \to 128$ + BN + ReLU | $16 \times 16 \times 128$ | 73,984 | 18,874,368 |
| MaxPool $2 \times 2$ | $8 \times 8 \times 128$ | 0 | – |
| Conv $3\times3$, $128 \to 256$ + BN + ReLU | $8 \times 8 \times 256$ | 295,424 | 18,874,368 |
| MaxPool $2 \times 2$; GAP | 256 | 0 | – |
| FC $256 \to 64$; ReLU; dropout | 64 | 16,448 | 16,384 |
| FC $64 \to 10$ | 10 | 650 | 640 |
| **Total** | | **405,994** | **60,179,072** |

For example, the first block has $9(3)(32) + 2(32) = 928$ parameters. The classifier contributes $(256 \cdot 64 + 64) + (64 \cdot 10 + 10) = 17,098$ parameters.

## *2.3 Model B: depthwise separable convolutions*

Each standard convolution is replaced by a $3 \times 3$ depthwise convolution (groups equal to input channels), BatchNorm, ReLU, a $1 \times 1$ pointwise convolution, BatchNorm and ReLU. Max-pooling positions and the classifier match Model A. This factorization follows the depthwise separable approach used in lightweight CNNs [2]. It includes an additional intermediate BatchNorm and ReLU, so the experiment compares two block designs rather than changing only a linear operator.

**Table 3:** Model B block parameters. Both BatchNorm layers are included in each block's parameter count.

| Block | DW weights | DW BN | PW weights | PW BN | Total |
|---|---|---|---|---|---|
| $3 \to 32$ | 27 | 6 | 96 | 64 | 193 |
| $32 \to 64$ | 288 | 64 | 2,048 | 128 | 2,528 |
| $64 \to 128$ | 576 | 128 | 8,192 | 256 | 9,152 |
| $128 \to 256$ | 1,152 | 256 | 32,768 | 512 | 34,688 |
| FC $256 \to 64$ | | | Weights and bias | | 16,448 |
| FC $64 \to 10$ | | | Weights and bias | | 650 |
| **Total** | | | | | **63,659** |

The four feature blocks in Table 3 require 503,808, 2,392,064, 2,244,608 and 2,170,880 MACs respectively. Adding 17,024 classifier MACs gives **7,328,384 MACs per image**. Model B uses

63.66% of the allowed parameter budget, leaving 36,341 parameters below the 100,000 limit. The summary script checks both hand-calculated totals against the constructed networks.

Ignoring normalization, the cost ratio for one separable convolution is

$$\frac{k^2 C_{\text{in}} + C_{\text{in}} C_{\text{out}}}{k^2 C_{\text{in}} C_{\text{out}}} = \frac{1}{C_{\text{out}}} + \frac{1}{k^2}.$$

For $k = 3$ and $C_{\text{out}} = 256$, this is approximately 0.115. Across the whole network, Model B has $6.38\times$ fewer trainable parameters and $8.21\times$ fewer counted MACs than Model A. Its parameter reduction is smaller than its MAC reduction because the shared classifier is unchanged.

## *2.4 Hardware-aware design choices*

ReLU computes $\max(0, x)$ with a comparison and avoids the exponentials used in sigmoid or softmax. It is straightforward to implement with integer arithmetic, although quantized execution still requires correct scale and zero-point handling. ReLU preserves a unit derivative for positive inputs, but negative inputs may have zero gradient; faster convergence is not guaranteed. The network outputs logits so inference needs only an argmax, without softmax probabilities.

GAP reduces the classifier input from $4 \times 4 \times 256 = 4,096$ values to 256. Flattening that tensor and using a 64-unit layer would require 262,208 parameters in that layer alone, exceeding Model B's entire budget. Max-pooling uses comparisons and reduces later activation sizes. Channel widths that are multiples of 32 are convenient for vectorized implementations, although their benefit depends on the target hardware.

BatchNorm can be folded into the preceding convolution for deployment. The supplied PyTorch timing code does not perform this folding, so measured latency includes the unfused modules. Model size on disk also does not measure peak RAM. A single $64 \times 64 \times 32$ feature tensor occupies 512 KiB in float32 or 128 KiB in int8; input tensors, intermediate buffers and operator workspace add to this amount. A small parameter count alone therefore does not prove that the complete network fits a particular microcontroller.

# 3. Optimizer Selection and Tuning

## *3.1 Experimental protocol*

The optimizer study trains Model B for 20 epochs per configuration with batch size 64. The same initial model weights are copied into every trial, and seed 42 is reset before each trial to control minibatch order and augmentation randomness. The study tests Adam at learning rates $10^{-2}$, $10^{-3}$ and $10^{-4}$, ordinary SGD at $10^{-2}$, and SGD at $10^{-2}$ with momentum $\mu \in \{0.5, 0.9, 0.99\}$. These trials use no learning-rate scheduler and no weight decay. We compare best validation accuracy, final validation loss, the standard deviation of validation loss over the last five epochs, and the first epoch at which validation accuracy reaches 85%.

For plain SGD, $\theta_{t+1} = \theta_t - \eta g_t$. With momentum,

$$v_t = \mu v_{t-1} + g_t, \qquad \theta_{t+1} = \theta_t - \eta v_t.$$

Consistently directed gradients accumulate in $v_t$, while alternating components may cancel. A large $\mu$ retains more history and can accelerate progress, but can also keep moving in an outdated direction. At fixed learning rate, increasing momentum changes the effective update magnitude as well as its smoothing.

Adam uses first- and second-moment estimates to adapt the step size of each parameter [3]. Its additional optimizer state affects training memory; it is not stored in the model-only checkpoints measured here and does not increase inference cost.

**Table 4:** Model B optimizer study, 20 epochs per run. $L_v$ is validation cross-entropy; $\sigma_{v,5}$ is its standard deviation over epochs 16–20. A dash means 85% was not reached.

| Optimizer | LR | Best val. (%) | Final $L_v$ | $\sigma_{v,5}$ | Epoch to 85% |
|---|---|---|---|---|---|
| Adam | $10^{-2}$ | 91.93 | 0.3233 | 0.2579 | 7 |
| Adam | $10^{-3}$ | 91.90 | 0.2365 | 0.0169 | 8 |
| Adam | $10^{-4}$ | 81.31 | 0.5264 | 0.0103 | – |
| SGD | $10^{-2}$ | 84.15 | 0.4425 | 0.2577 | – |
| SGD ($\mu = 0.5$) | $10^{-2}$ | 88.02 | 0.9693 | 0.2307 | 15 |
| SGD ($\mu = 0.9$) | $10^{-2}$ | 92.96 | 0.4642 | 0.1209 | 6 |
| SGD ($\mu = 0.99$) | $10^{-2}$ | 91.01 | 0.2837 | 0.0598 | 9 |

## *3.2 Selected optimizer and learning rate*

The final custom-model training uses Adam with initial learning rate $10^{-3}$. Table 4 supports this as a stable configuration, but not as the trial with the highest validation accuracy. SGD with momentum 0.9 reaches **92.96%** best validation accuracy, exceeding Adam at $10^{-3}$ (91.90%), and reaches 85% in six epochs instead of eight. Adam at $10^{-3}$ nevertheless has the lowest final

validation loss, 0.2365, and a much smaller late-epoch loss variation, 0.0169 versus 0.1209 for SGD with momentum 0.9. Figure 2 shows these differences.

Adam at $10^{-2}$ has almost the same peak validation accuracy as $10^{-3}$ (91.93% versus 91.90%) but a much larger loss standard deviation of 0.2579. Adam at $10^{-4}$ reaches only 81.31% within 20 epochs. The selected $10^{-3}$ therefore balances progress and observed stability over this training budget. This is an interpretation of the measured comparison; the pipeline fixes Adam at $10^{-3}$ in its final training script rather than automatically selecting an optimizer from the study output.

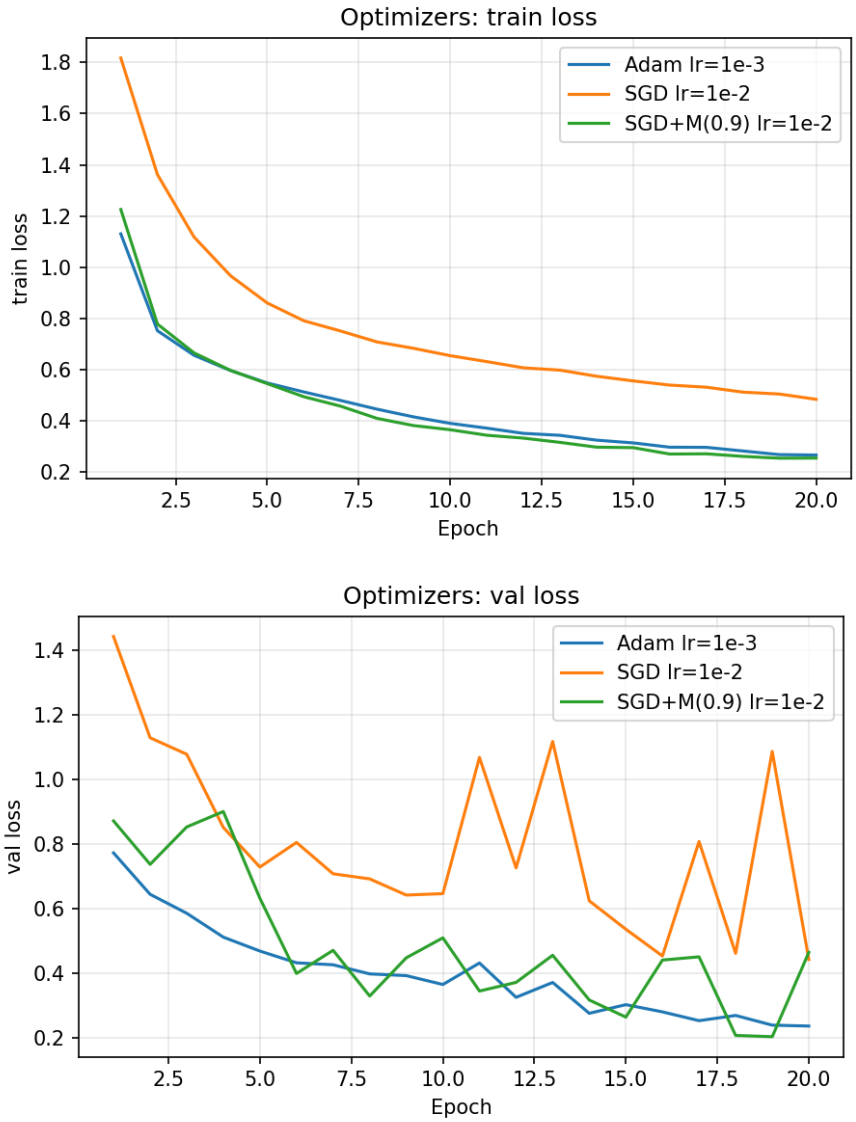



**Figure 2:** Training and validation loss for Adam, plain SGD and SGD with momentum 0.9. Original recorded plot panels are stacked for legibility.

## *3.3 Learning-rate sensitivity and momentum*

The learning-rate sweep in Figure 3 shows that the largest Adam step size can reduce training loss quickly while producing substantial fluctuations in validation loss. The smallest step size

leaves both losses relatively high at the end of the fixed budget. This illustrates why a low training loss alone is insufficient to justify the chosen learning rate.

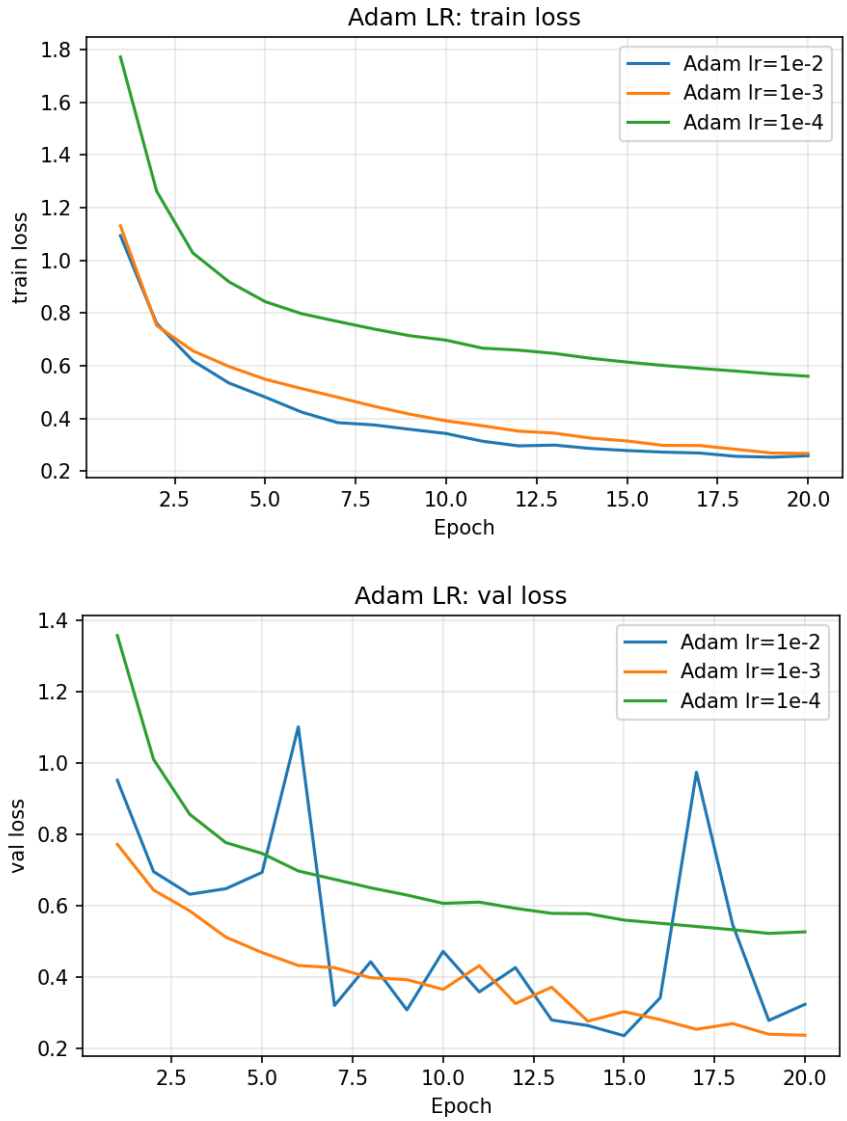



**Figure 3:** Adam learning-rate sweep on Model B. All trials use the same initial weights and 20-epoch budget.

Plain SGD reaches 84.15% best validation accuracy and does not reach the 85% target. Momentum 0.5 increases the best accuracy to 88.02%, reaching the target at epoch 15. Momentum 0.9 gives the strongest peak performance and fastest target attainment among the tested momentum values. Increasing momentum further to 0.99 reduces the best accuracy to 91.01% and delays the target to epoch 9. More momentum does not monotonically improve convergence.

Figure 4 shows fluctuations in the validation trajectories. However, the last-five-epoch loss variation for momentum 0.99 is actually lower than for 0.9 (0.0598 versus 0.1209). The data do not support a blanket claim that 0.99 is the most oscillatory choice. Its retained gradient history and the fixed step size are plausible contributors to its lower accuracy, but this study does not

isolate that mechanism.

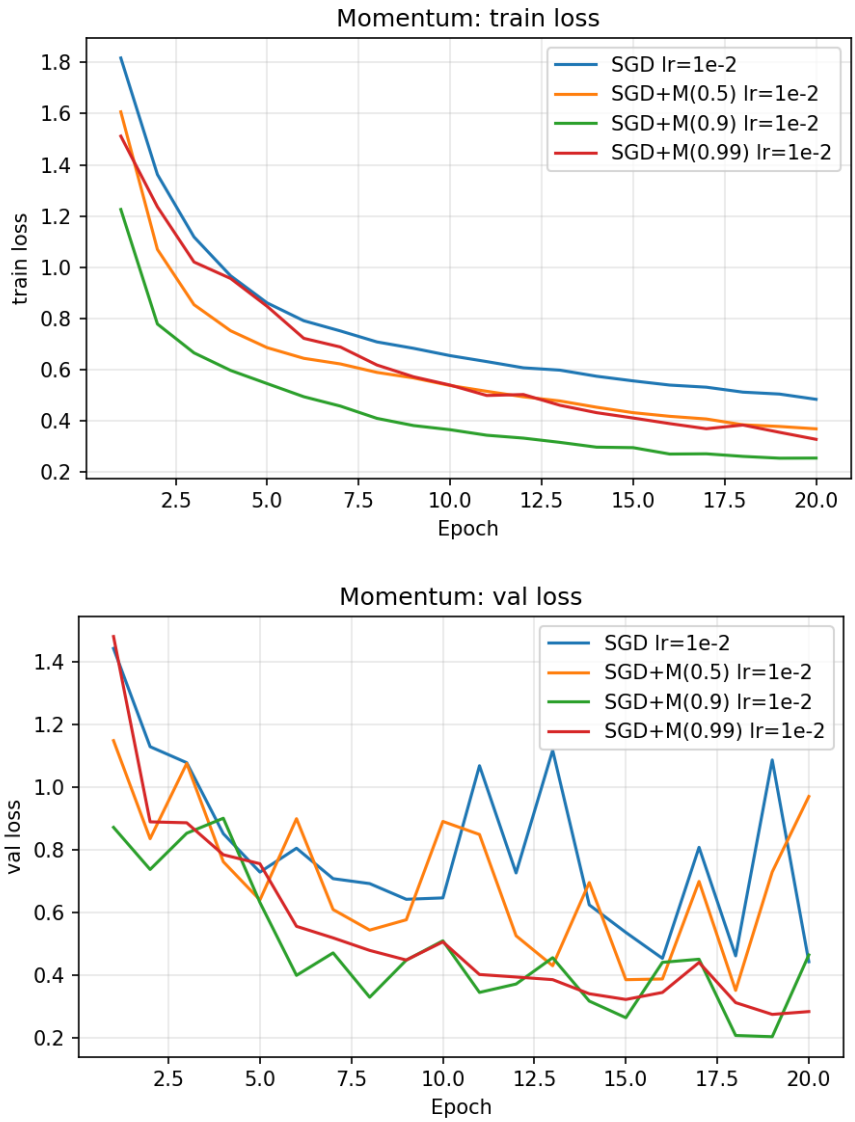



**Figure 4:** Effect of momentum with SGD learning rate fixed at $10^{-2}$.

## *3.4 Scope of the optimizer comparison*

The final training configuration adds weight decay $10^{-4}$, cosine learning-rate decay and ten extra epochs. Improvements from the optimizer study to the final model cannot therefore be attributed solely to Adam or to its learning rate. The study also evaluates the test set after each optimizer trial and stores those scores. This exposes the test set during experimentation; an improved protocol would reserve it for one evaluation after all choices are finalized. The selection discussion here uses validation results, and the reported test scores are descriptive results from the recorded workflow. No repeated-seed experiment was performed to quantify variation.

# 4. Custom Model Training and Evaluation

## *4.1 Training settings and checkpoint selection*

Both custom networks are trained from scratch for **30 epochs**, exceeding the required 20 epochs. Each uses cross-entropy, batch size 64, Adam with initial learning rate $10^{-3}$ and weight decay $10^{-4}$. A cosine schedule reduces the learning rate toward zero over 30 epochs. Model initialization is seeded with 42 for each network. The architectures consume random numbers differently during initialization, so resetting this seed does not guarantee identical minibatch or augmentation sequences across Model A and Model B.

The training loop records sample-weighted mean losses and accuracies each epoch. After completing the full budget, it restores the checkpoint with the highest validation accuracy and evaluates that checkpoint on the test set. The selected epochs are 28 for Model A and 27 for Model B. Table 5 includes all four models for reference; training minutes sum training-only epoch durations and exclude validation, downloads and plotting.

**Table 5:** Training budgets and checkpoint selection for the four final models.

| Model | Epochs | Best epoch | Best val. (%) | Train min. |
|---|---|---|---|---|
| Model A | 30 | 28 | 97.01 | 7.82 |
| Model B | 30 | 27 | 94.20 | 8.08 |
| MobileNetV2 | 20 | 15 | 98.10 | 10.06 |
| ShuffleNetV2 | 20 | 19 | 97.85 | 10.79 |

The recorded run used CUDA on the VM. The supplied machine information identifies an NVIDIA Quadro M6000 with approximately 11.25 GiB of visible device memory and driver 525.89.02. The configured environment used Python 3.10 with PyTorch 2.2.2 and torchvision 0.17.2 (CUDA 11.8 builds). Exact installed package versions were not preserved in the transferred environment file, which is empty; these configured versions should not be mistaken for a complete environment lock.

## *4.2 Loss curves and convergence*

Model A's training loss falls from 0.9338 to 0.0887; its final validation loss is 0.0901. Model B's training loss falls from 1.1529 to 0.2043, with final validation loss 0.1735. Figures 5 and 6 show the complete trajectories. The end-of-training losses suggest a capacity or optimization limitation in Model B relative to Model A, rather than severe final overfitting in Model A. This is a descriptive reading of one run rather than a causal diagnosis.

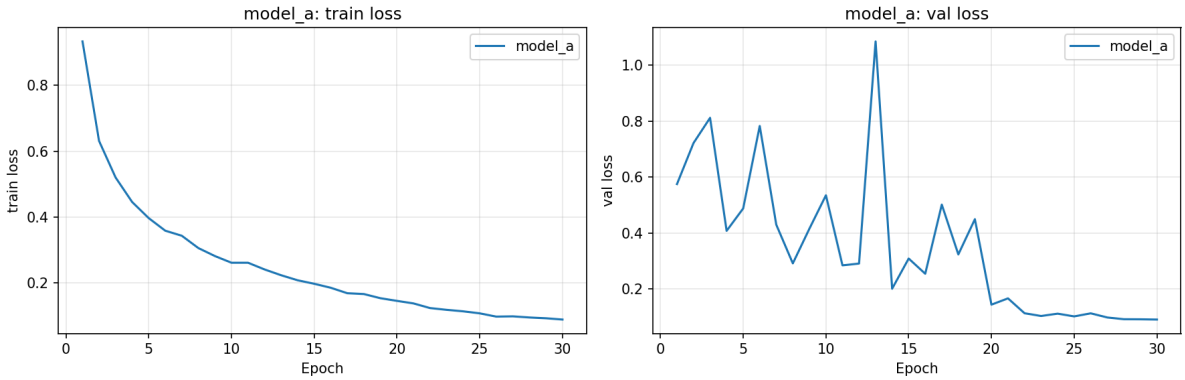



**Figure 5:** Model A training and validation cross-entropy over 30 epochs. Test metrics use the best validation checkpoint, not necessarily the final epoch.

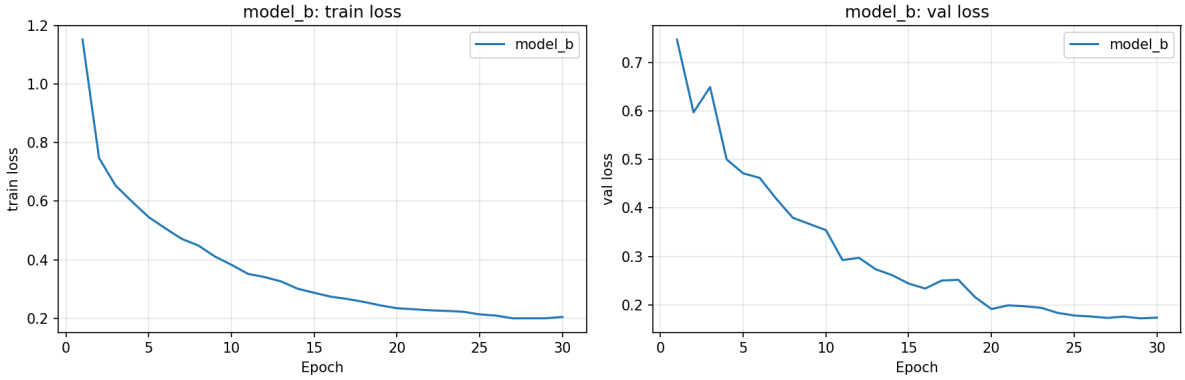



**Figure 6:** Model B training and validation cross-entropy over 30 epochs.

Model B's validation loss can be below its training loss because training includes random augmentation and dropout, whereas evaluation disables these. BatchNorm also behaves differently in training and evaluation modes. A lower validation loss is therefore not, by itself, evidence of a data error.

## *4.3 Metrics and the custom-model comparison*

Accuracy is the number of correct predictions divided by 4,050 test images. For each class $c$,

$$\text{Precision}_c = \frac{TP_c}{TP_c + FP_c}, \qquad \text{Recall}_c = \frac{TP_c}{TP_c + FN_c}.$$

Macro precision and recall are the arithmetic means of the ten class-specific values. Every test class has positive support. The matrices below use true classes as rows and predicted classes as columns.

**Table 6:** Required Model A versus Model B comparison, using measured float32 checkpoint sizes.

| Model | Parameters | Size (KiB) | Train s / epoch | Test accuracy (%) |
|---|---|---|---|---|
| Model A | 405,994 | 1,598.7 | 15.63 | 97.11 |
| Model B | 63,659 | 270.9 | 16.17 | 94.54 |

In Table 6, Model A correctly classifies 3,933 images and makes 117 errors. Model B correctly classifies 3,829 and makes 221 errors: 104 additional errors, or a 2.57 percentage-point accuracy reduction. Table 7 shows the class-level effect. Model B is strongest on SeaLake and Forest; its lowest recall is PermanentCrop at 88.00%, followed by Highway at 89.60%.

**Table 7:** Custom-model per-class precision (P) and recall (R), in percent.

| Class | Model A P (%) | Model A R (%) | Model B P (%) | Model B R (%) |
|---|---|---|---|---|
| AnnualCrop | 95.12 | 95.33 | 92.32 | 93.56 |
| Forest | 98.25 | 100.00 | 98.02 | 99.11 |
| HerbaceousVegetation | 96.44 | 96.44 | 92.67 | 92.67 |
| Highway | 98.11 | 96.80 | 92.82 | 89.60 |
| Industrial | 97.63 | 98.67 | 96.26 | 96.00 |
| Pasture | 96.38 | 97.67 | 94.12 | 96.00 |
| PermanentCrop | 94.65 | 94.40 | 90.41 | 88.00 |
| Residential | 98.89 | 99.11 | 96.11 | 98.89 |
| River | 95.12 | 93.60 | 91.94 | 91.20 |
| SeaLake | 99.77 | 98.44 | 99.33 | 98.67 |
| Macro mean | 97.04 | 97.05 | 94.40 | 94.37 |

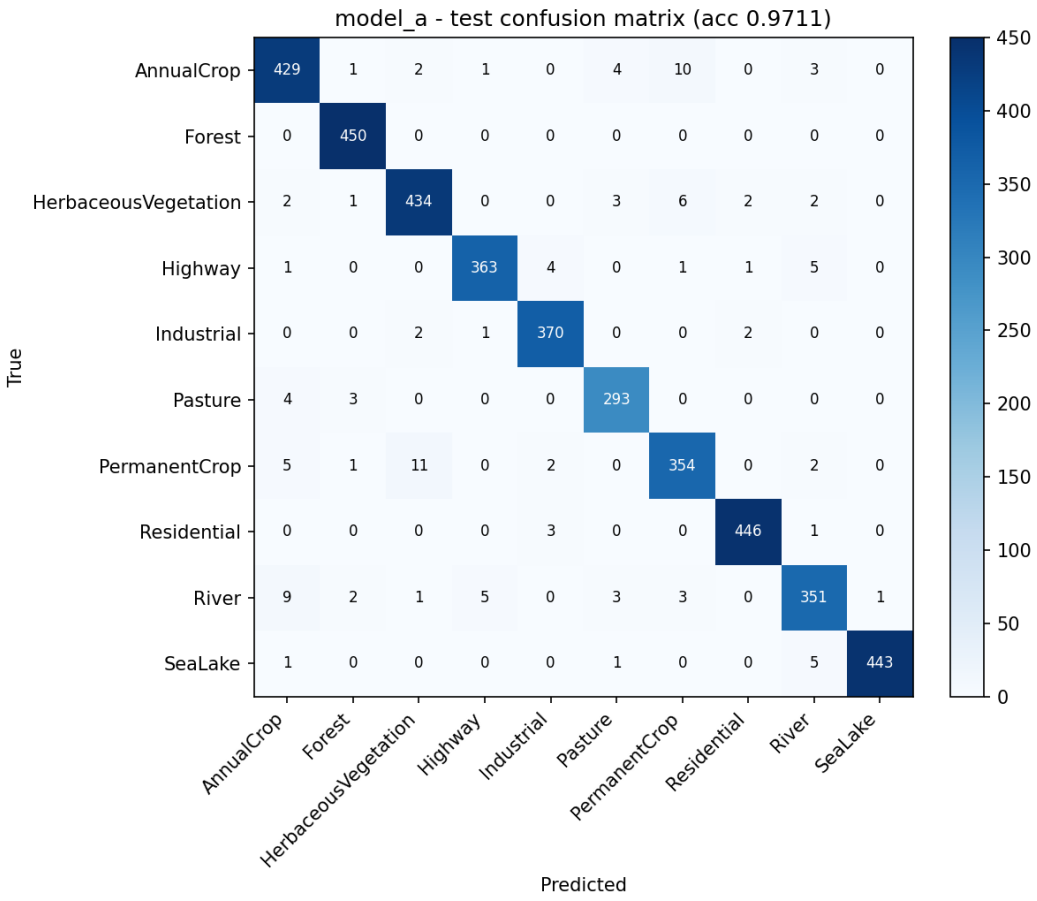



**Figure 7:** Model A test confusion matrix, 4,050 images.

Model A's largest single off-diagonal count is 11 PermanentCrop images predicted as HerbaceousVegetation. Ten AnnualCrop images are predicted as PermanentCrop, and nine River images as AnnualCrop. Most remaining mistakes are concentrated in a few class pairs rather than spread evenly across all classes (Figure 7).

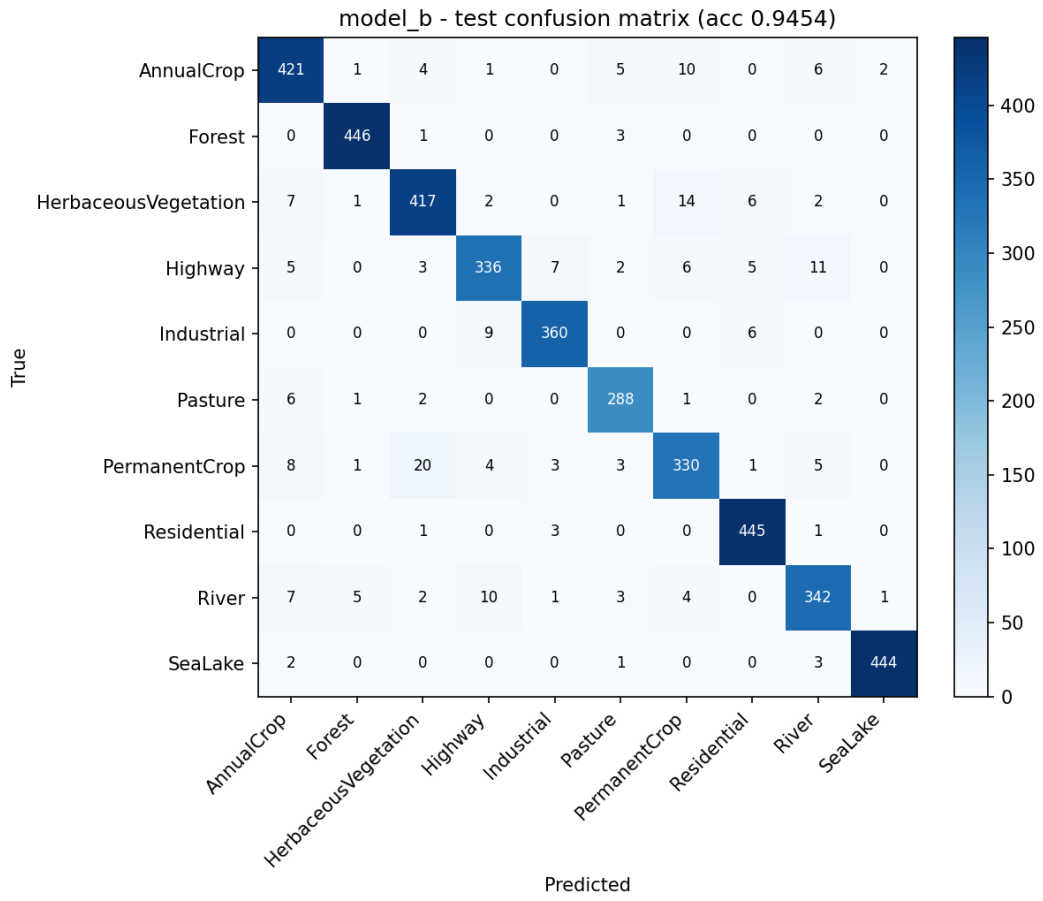



**Figure 8:** Model B test confusion matrix, using exactly the same test indices as Model A.

Model B confuses 20 PermanentCrop images with HerbaceousVegetation and makes 14 errors in the reverse direction. Highway and River are confused 11 and ten times respectively (Figure 8). Similar vegetation textures and narrow linear structures are plausible explanations, but confirming the visual causes would require inspecting the misclassified images themselves.

## *4.4 Effect of depthwise separation*

Model B reduces the measured checkpoint size by 83.05% (a $5.90\times$ reduction), and counted MACs by 87.82%. However, mean training time increases slightly from 15.63 to 16.17 seconds per epoch, about 3.43%. Depthwise operations can have lower arithmetic intensity and incur additional intermediate tensors and operator launches. These are plausible explanations for the observed timing, not measurements from a kernel profiler. Fewer MACs do not guarantee a proportional reduction in wall-clock time.

Single-thread CPU latency falls from 6.71 to 5.62 ms per image, about 16.2%. The benchmark reports the median of 100 batch-one forward passes after ten warm-ups. It measures unfused float32 execution on the VM's CPU, excluding preprocessing. These timings are comparative measurements on this machine, not microcontroller deployment guarantees.

# 5. Lightweight SOTA Fine-Tuning and Evaluation

## *5.1 Architectures and fine-tuning procedure*

We choose two architectures designed for mobile or efficient inference. MobileNetV2 uses inverted residual blocks with depthwise filtering and linear bottlenecks [4]. ShuffleNetV2 uses channel splitting and shuffling as part of an architecture designed with practical execution efficiency in mind [5]. The latter paper's emphasis on actual runtime, alongside operation counts, is relevant to the timing differences observed here.

Both torchvision models load ImageNet pretrained weights (the `IMAGENET1K_V1` weight enums). MobileNetV2's final classifier is replaced by a $1280 \to 10$ linear layer, and ShuffleNetV2 $\times 1.0$ receives a $1024 \to 10$ linear layer. All parameters are fine-tuned; there is no frozen-backbone stage. The resulting trainable parameter counts equal the total parameter counts: 2,236,682 and 1,263,854 respectively.

Both models train for 20 epochs with batch size 64, AdamW and weight decay $10^{-4}$. The backbone starts at learning rate $10^{-4}$, and the new classifier starts at $10^{-3}$. A cosine schedule decays both learning rates. This higher classifier learning rate allows the randomly initialized head to adapt more quickly while applying smaller initial changes to pretrained features. Each model receives the same saved train, validation and test indices, normalization and training augmentation as the custom networks, with $64 \times 64$ input throughout.

## *5.2 Test performance and learning curves*

Table 8 summarizes final test performance for all networks. MobileNetV2 reaches 98.49% accuracy with 61 errors, and ShuffleNetV2 reaches 97.80% with 89 errors. Compared with Model B, these correspond to 160 and 132 fewer mistakes on this test set. Macro precision and recall improve consistently, so the accuracy gains are not explained only by the largest classes.

**Table 8:** Best-checkpoint test performance. Precision and recall are macro averages.

| Model | Accuracy (%) | Precision (%) | Recall (%) | Errors / 4,050 |
|---|---|---|---|---|
| Model A | 97.11 | 97.04 | 97.05 | 117 |
| Model B | 94.54 | 94.40 | 94.37 | 221 |
| MobileNetV2 | 98.49 | 98.48 | 98.38 | 61 |
| ShuffleNetV2 | 97.80 | 97.75 | 97.70 | 89 |

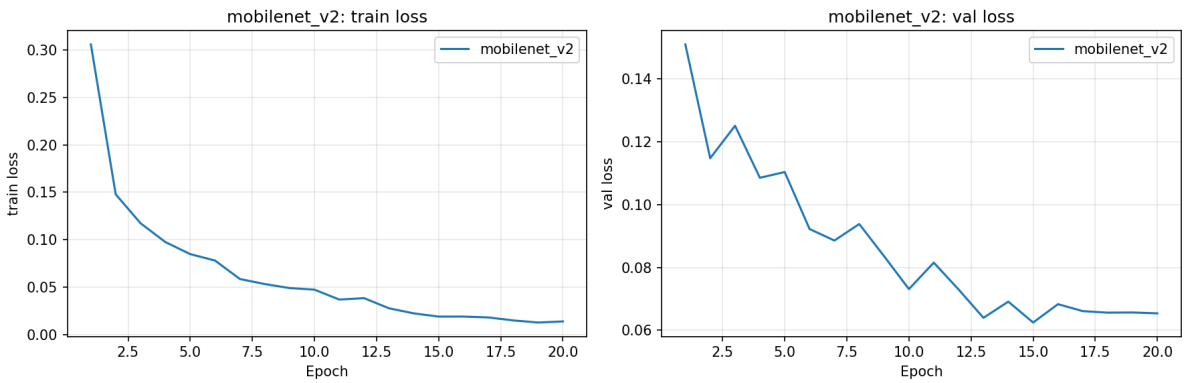



**Figure 9:** MobileNetV2 fine-tuning loss over 20 epochs. The selected checkpoint is from epoch 15.

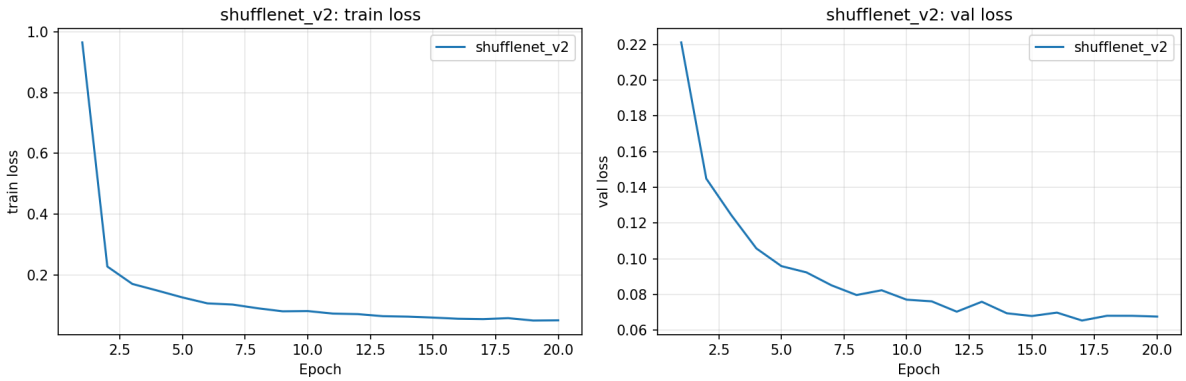



**Figure 10:** ShuffleNetV2 fine-tuning loss over 20 epochs. The selected checkpoint is from epoch 19.

MobileNetV2 finishes with training loss 0.0139 and validation loss 0.0654, while ShuffleNetV2 finishes with 0.0508 and 0.0677 respectively. The larger train–validation gap in MobileNetV2 (Figure 9) is consistent with some additional fitting of the training set. Selecting epoch 15 by validation accuracy avoids assuming that the lowest final training loss provides the best checkpoint. ShuffleNetV2's selected checkpoint is closer to the end of its budget (Figure 10).

**Table 9:** Pretrained-model per-class precision (P) and recall (R), in percent.

| Class | MobileNetV2 P (%) | R (%) | ShuffleNetV2 P (%) | R (%) |
|---|---|---|---|---|
| AnnualCrop | 95.48 | 98.67 | 96.67 | 96.89 |
| Forest | 100.00 | 99.78 | 99.12 | 99.78 |
| HerbaceousVegetation | 98.22 | 98.22 | 95.40 | 96.89 |
| Highway | 97.89 | 98.93 | 97.57 | 96.53 |
| Industrial | 99.20 | 98.93 | 98.15 | 98.93 |
| Pasture | 97.32 | 96.67 | 96.67 | 96.67 |
| PermanentCrop | 98.89 | 94.93 | 96.69 | 93.60 |
| Residential | 99.12 | 99.56 | 99.33 | 99.33 |
| River | 98.92 | 98.13 | 98.14 | 98.40 |
| SeaLake | 99.78 | 100.00 | 99.78 | 100.00 |
| Macro mean | 98.48 | 98.38 | 97.75 | 97.70 |

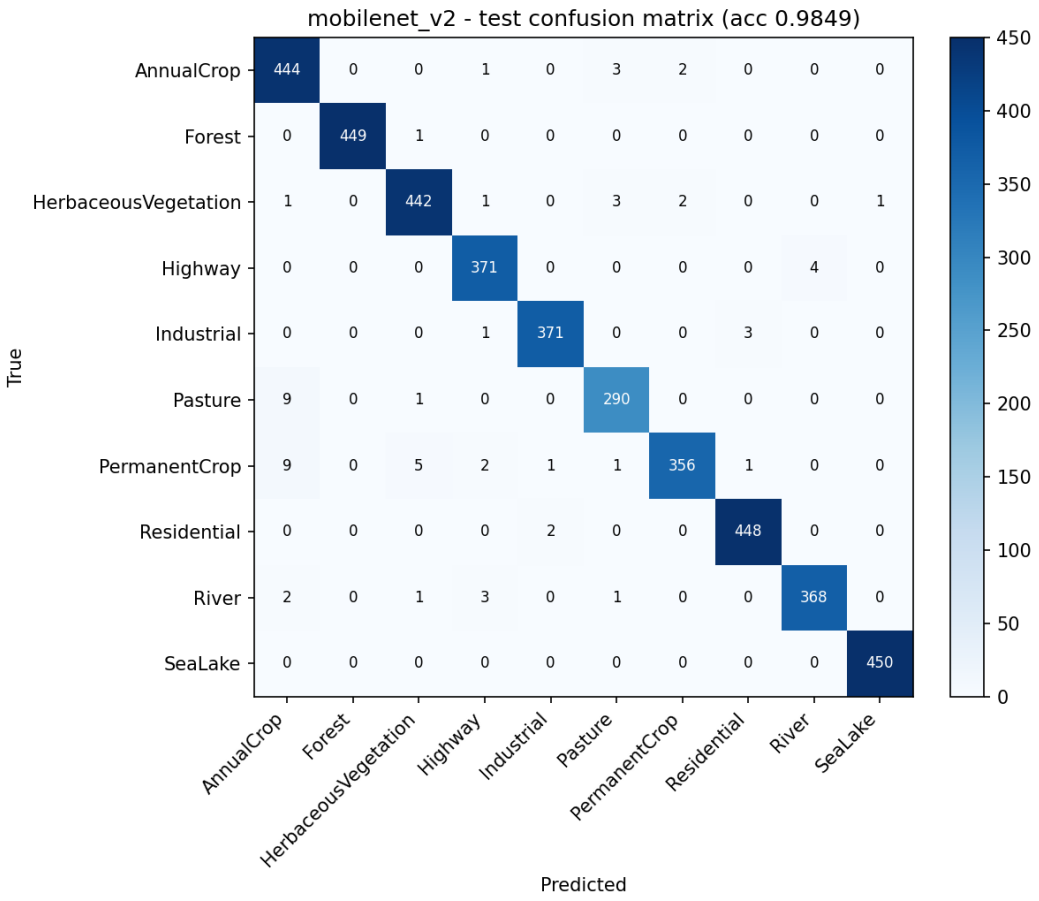



**Figure 11:** MobileNetV2 test confusion matrix after fine-tuning.

In Figure 11, nine PermanentCrop and nine Pasture images are predicted as AnnualCrop, the largest off-diagonal counts. Table 9 shows that PermanentCrop recall remains the lowest at 94.93%. The perfect SeaLake recall applies to its 450 test images only and should not be interpreted as universal error-free performance.

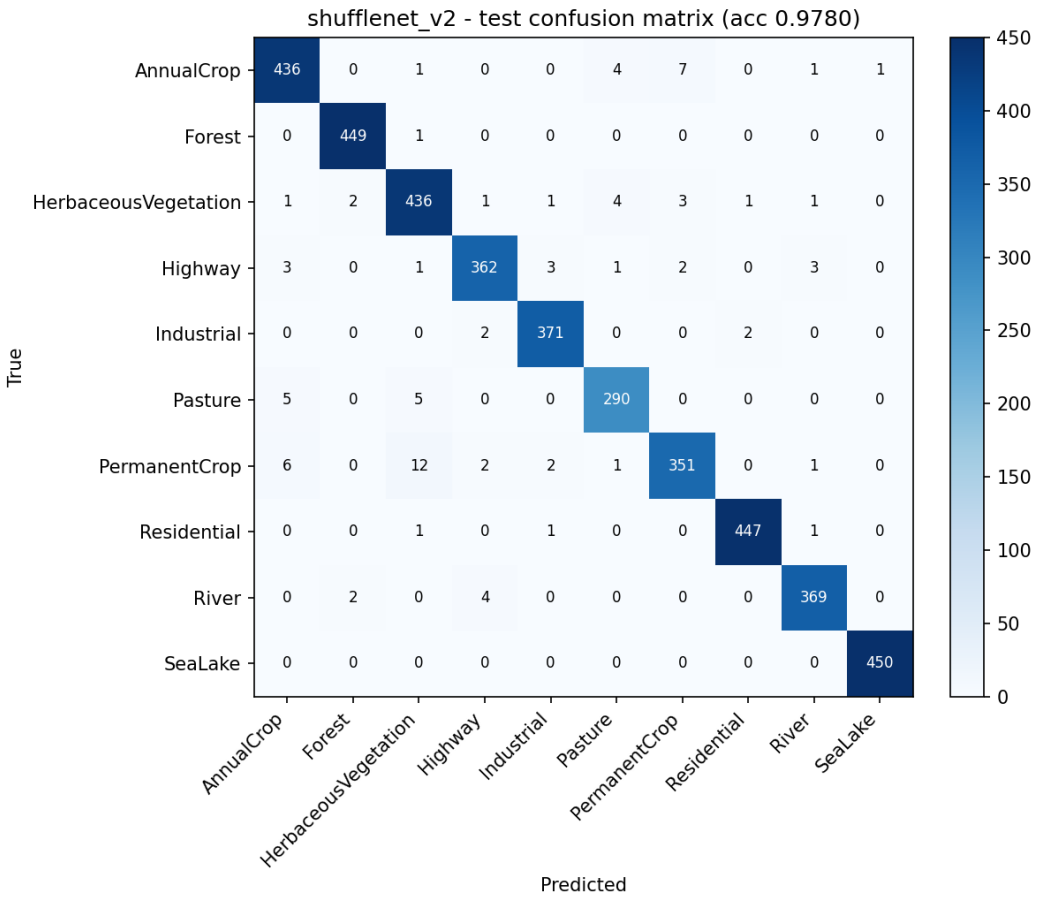



**Figure 12:** ShuffleNetV2 test confusion matrix after fine-tuning.

ShuffleNetV2's largest error count is 12 PermanentCrop images predicted as HerbaceousVegetation, followed by seven AnnualCrop images predicted as PermanentCrop (Figure 12). These overlaps persist across different architectures, suggesting that the class boundaries themselves present difficulty at this resolution. The images would need direct inspection to establish the source of each error.

# 6. Final Comparison and Trade-off Discussion

## *6.1 Accuracy, storage and computational cost*

Table 10 lists all resource measurements, and Figure 13 relates test accuracy to checkpoint size. Storage units are binary: 1 KiB $= 1024$ bytes and 1 MiB $= 1024^2$ bytes. The pipeline labels these quantities "KB" and "MB"; this report uses the precise unit names. Sizes refer to serialized float32 state dictionaries, including BatchNorm buffers and serialization overhead. They exclude optimizer state, the Python runtime and the inference engine.

**Table 10:** Resources at $64 \times 64$ input. CPU latency is single-thread, batch one; training time is the mean training-only epoch duration on the GPU.

| Model | Parameters | M MACs | MiB | s / epoch | CPU ms |
|---|---|---|---|---|---|
| Model A | 405,994 | 60.18 | 1.561 | 15.63 | 6.71 |
| Model B | 63,659 | 7.33 | 0.265 | 16.17 | 5.62 |
| MobileNetV2 | 2,236,682 | 24.46 | 8.767 | 30.17 | 13.68 |
| ShuffleNetV2 | 1,263,854 | 11.76 | 4.988 | 32.36 | 14.34 |

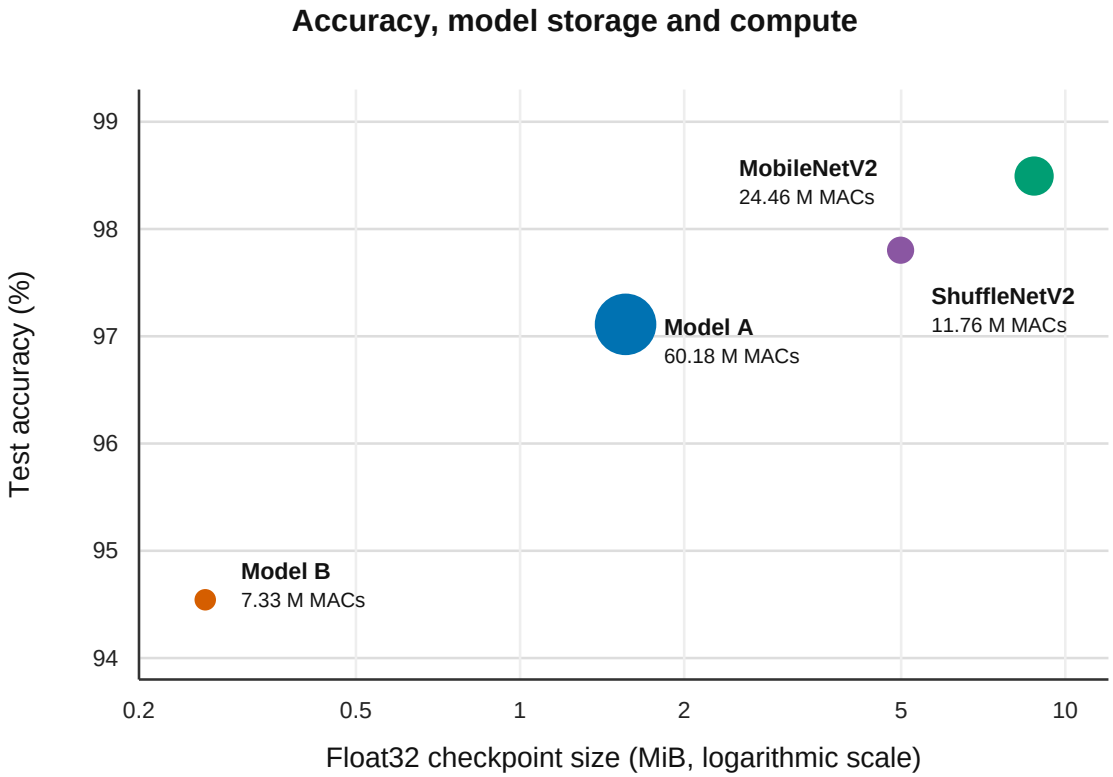



**Figure 13:** Measured accuracy versus float32 checkpoint size. Bubble area is proportional to convolutional and linear MACs; labels show millions of MACs.

Model B achieves 94.54% accuracy with a 0.265 MiB checkpoint. MobileNetV2 adds 3.95 per-

centage points of accuracy but uses approximately $33.13\times$ the checkpoint storage and $3.34\times$ the MACs. ShuffleNetV2 adds 3.26 points while using $18.85\times$ the storage and $1.60\times$ the MACs. These are substantial storage increases even though both pretrained networks are lightweight relative to larger conventional vision backbones.

The parameter ratios differ from the file-size ratios: MobileNetV2 has $35.14\times$ as many trainable parameters as Model B and ShuffleNetV2 has $19.85\times$. File overhead and non-parameter buffers account for part of the difference. Raw float32 parameter storage for Model B is $63,659 \cdot 4/1024 = 248.67$ KiB, smaller than the measured 270.93 KiB state dictionary. Using parameter bytes alone would understate the file that was actually saved.

## *6.2 Why smaller operation counts do not settle the choice*

On the VM, Model B has the lowest measured CPU latency at 5.62 ms; MobileNetV2 and ShuffleNetV2 take 13.68 and 14.34 ms respectively. ShuffleNetV2 therefore has lower MACs than MobileNetV2 but slightly higher observed CPU latency and training time. Operator composition, tensor movement and implementation efficiency affect execution. This is consistent with the need to measure speed on the intended hardware rather than ranking architectures using MACs alone [5].

Model A provides an intermediate point: 97.11% accuracy and a 1.561 MiB checkpoint, although its 60.18 million MACs are the highest in the comparison. Its standard convolutions may execute efficiently on the tested runtime despite their high operation count. Depending on the deployment constraint, it can be a reasonable compromise even though Model B is the only custom model that satisfies the sub-100k budget.

## *6.3 Custom training versus pretrained networks*

The custom design offers direct control over parameter count, channels, classifier size and operator selection. It is the most storage-efficient option in this experiment and can be adapted to a fixed device budget. Its cost is lower predictive performance: Model B's restricted capacity and training from scratch leave more difficult crop and linear-structure cases unresolved.

Pretraining provides reusable visual features and allows the larger mobile networks to achieve better accuracy in the 20-epoch fine-tuning budget. However, architecture, parameter count, initialization, optimizer and training budget all differ from the custom runs. The accuracy gain cannot be assigned exclusively to pretraining without an additional comparison that trains the same SOTA architecture from scratch. The pretrained models require more persistent weight storage, and their runtime advantage on a phone cannot be assumed on a small embedded CPU.

For a device with a strict sub-100k parameter requirement, Model B is the feasible choice among these four. If several MiB of model storage are available and classification accuracy is the priority, MobileNetV2 gives the best measured result. ShuffleNetV2 reduces storage relative

to MobileNetV2 at a 0.69 percentage-point accuracy cost, but its runtime must be measured on the actual target. These choices follow the measured constraints rather than a single universal ranking.

## *6.4 Deployment and experimental limitations*

An int8 representation of Model B's parameters alone would occupy approximately 62.17 KiB. This is an estimate: no quantization, embedded export or hardware deployment is performed in this run. Quantization may alter accuracy, and scale metadata, buffers, operator workspace and executable code require additional memory. The current float32 latency is not a measurement of an optimized int8 implementation.

The experiments use one seed and one random image split. They provide no confidence interval over repeated training, no geographic holdout, no energy measurement and no peak-RAM measurement. The optimizer study exposes test scores during tuning, as discussed in Section 3.4. Cross-model initializations do not guarantee identical augmentation streams, and the pretrained models have a different optimization protocol. These limits should guide how strongly the observed differences are generalized.

The principal measured result is that depthwise separation produces a much smaller custom model with fewer MACs, at a moderate accuracy cost, while the pretrained mobile networks recover accuracy at a large increase in stored model size. Training-time savings do not follow the MAC reduction on this GPU, and deployment decisions require measurements beyond trainable parameter count.

# A. Reproducibility and Result Provenance

The code is available at `https://github.com/Parawork/CNN-for-image-classification`, with the contributor profile linked on the cover. The inspected source revision is `09873fd`; the Git history records work from September 21 to October 17, 2026. The transferred full-run log records successful completion of all six pipeline stages in 1 hour, 25 minutes and 19 seconds. That duration includes work beyond the training-only times in Table 5.

The six steps prepare the dataset, verify custom model complexity, run seven optimizer trials, train the two custom models, fine-tune the two pretrained networks and assemble comparison tables. The full run uses the default epoch budgets; the two-epoch quick mode is not the source of this report. The command used after dataset availability was established was:

```
python run_all.py --from 1
```

The following saved artifacts support the report:

- `data/splits.json`: the shared split indices.
- `outputs/results/dataset_summary.json`: dataset counts.
- `outputs/results/model_summary.json`: parameters and MACs.
- `outputs/results/optimizer_study_model_b.json`: all seven trials.
- Model-specific JSON files in `outputs/results/`: histories, settings, metrics, confusion matrices and sizes.
- `outputs/results/comparison.md`: comparison tables and CPU latency values.
- `outputs/checkpoints/`: selected model state dictionaries.
- `outputs/figures/`: experiment plots reproduced in this report.

Report preparation checks confirm the split sizes and disjointness, all four epoch counts, the 64-pixel inputs, test-set supports and accuracies reconstructed from the confusion matrices. Macro precision and recall are also reconstructed from those matrices and checked against the saved values. Source hashes and these checks are recorded in `validation.json` alongside the report source. The checks validate consistency of the transferred artifacts; they do not constitute a new training run.

To rebuild the document, regenerate its tables with `build_assets.py` and compile `main.tex` using XeLaTeX. XeLaTeX is used to match the Times New Roman and Consolas fonts in the previous assignment report. The report filename uses the team name PatternX because no numerical group identifier was assigned. Code is to be submitted separately from the PDF as required by the assignment.

# References


[1] P. Helber, B. Bischke, A. Dengel and D. Borth, "EuroSAT: A Novel Dataset and Deep Learning Benchmark for Land Use and Land Cover Classification," 2019. https://arxiv.org/abs/1709.00029.

[2] A. G. Howard et al., "MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications," 2017. https://arxiv.org/abs/1704.04861.

[3] D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," ICLR, 2015. https://arxiv.org/abs/1412.6980.

[4] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov and L.-C. Chen, "MobileNetV2: Inverted Residuals and Linear Bottlenecks," CVPR, 2018. https://arxiv.org/abs/1801.04381.

[5] N. Ma, X. Zhang, H.-T. Zheng and J. Sun, "ShuffleNet V2: Practical Guidelines for Efficient CNN Architecture Design," ECCV, 2018. https://arxiv.org/abs/1807.11164.